

consistency data queries may not have the most recent updates because it takes time for updates to reach replicas across a database cluster.

Isolation

Usually, many transactions run in parallel. Isolation guarantees that the intermediate state of a single transaction is invisible to other transactions, leaving the database in the same state that would have been obtained if these transactions were executed sequentially. As a result, transactions that run concurrently appear to be executed in a serialised or sequential order.

For example, in an application that transfers funds from one account to another, the isolation property ensures that another transaction sees the funds transferred from one account to the other, not leaving both the accounts without credits or debits.

Isolation properties can face a problem due to concurrency, called ‘read phenomena’. This is when a transaction reads data that another transaction may have changed.

- **Dirty reads:** Reading data that is not fully committed.
- **Non-repeatable reads:** Reading the same data twice. The query could be different, but the same data is read. For example, if one reads the sum of picture likes and that of the users who liked the picture, the values would have changed due to other transactions. This leads to a mismatch between the sum of likes from the user table and the total number of likes from the pictures table.
- **Phantom reads:** Re-reading non-existence data, such as reading the sum of likes; then a new record gets inserted by another transaction and that is not read.
- **Lost updates/serialisation anomaly:** Reading the written data, but unable to find it because other transactions have been deleted or modified.

Isolation levels define the degree to which a transaction must be isolated from the data modifications made by any other transaction in the database system. A transaction isolation level is defined by the above phenomena.

There are different types of isolation.

- **Read uncommitted isolation:** Allows to read uncommitted changes across transactions. This will be fast as nothing is to be maintained. However, as a downside, there will be several dirty reads happening.
- **Read committed isolation:** Most popular and default isolation levels, while running a transaction, can only see the changes committed by other transactions.
- **Repeatable read isolation:** When transactions read a row, they will remember it and keep it unchanged. This remembered value remains the same as the one read before; hence, when re-read happens, it is not affected. The caveat here is that this is expensive, and comes at the cost of valid reads.

- **Serializable isolation:** All the transactions run here serially, losing the concept of concurrency. This is the slowest of all.
- **Snapshot isolation:** Each transaction takes the snapshot of the committed changes before the start of the transaction. Here the new inserted rows that cause phantom reads will not occur as these get read from the snapshot. If the number of concurrent transactions increases, it would be costly in terms of the space holding N number of snapshots.

Durability

All the successful transaction information needs to be stored in persistent storage (non-volatile system storage), so that even in the event of a system failure such as hardware crash or power failure, we can recover the committed changes.

To make writing faster, some databases tend to write into RAM and in a backend batch-wise snapshot to the disk. While this is faster, it is also less durable. For example, in an application that transfers funds from one account to another, the durability property ensures that the changes made to each account have made persistent changes irreversible.

Here are a few techniques that ensure high durability.

- **WAL - Write ahead log**
 - Writing in the direct database is expensive and also risky when performing uncommitted changes. This is because during a failure you need to reverse everything.
 - WAL records the changes in the log in stable storage and later transfers to the database.
 - There is always an issue with OS cache, which writes in RAM and transfers to the hard disk batchwise; we need to record every item in a persistent database. Calling the `fsync OS` command resolves this, as it forces writes to the disk.
 - Checkpoint is a point where the program writes all the changes specified in the WAL to the database and then clears the logs.
- **Snapshot**
 - Generates complete snapshots of the data set in the memory within the specified time interval, and then saves it in the hard disk.
 - The save operation can be synchronous or asynchronous.
 - There are primarily two issues with this technique. First, the last in-memory data may be lost. Because there is a time interval for generating snapshots, it will cause the last data to fail to generate a snapshot. Second, it is time-consuming and performance-consuming, because the data in the memory needs to be completely snapshotted and written to the hard disk.